Credit Card Fraud Detection

Authors: Daniel Kuris, Noam Galinsky

Overview

This project investigates credit card fraud detection using Kaggle's Credit Card Fraud Detection dataset.

Dataset:
https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud/data

The project compares several machine learning models and explores techniques for handling highly imbalanced data, understanding feature importance, and evaluating model performance under different training conditions.

Models Evaluated

- K-Nearest Neighbors (KNN)
- Logistic Regression
- AdaBoost
- Random Forest

Research Questions

This project aims to answer the following questions:

1. **How should severe class imbalance be handled?**
   - Compare multiple balancing techniques to determine which produces the best fraud detection performance.
2. **How does the transaction amount affect fraud detection?**
   - Analyze the influence of the `Amount` feature on model predictions.
3. **Which features contribute most to fraud detection?**
   - Identify the most influential hidden PCA features (V1-V28).
4. **How much training data is actually required?**
   - Evaluate model performance using different training set sizes to identify when performance saturates.
5. **Does fraud detection performance change over time?**
   - Split the dataset into temporal segments and examine model stability across different time periods.

Dataset Overview

The dataset contains **284,807** credit card transactions made by European cardholders.

- **Fraud transactions:** 492 (0.172%)
- **Normal transactions:** 284,315

This makes the dataset **extremely imbalanced**, making fraud detection particularly challenging.

Features

Feature	Description
Time	Seconds elapsed since the first transaction

Feature	Description
Amount	Transaction amount
V1-V28	PCA-transformed confidential features
Class	Target variable (0 = legitimate, 1 = fraud)

Because the original transaction information is confidential, the majority of features are anonymized through Principal Component Analysis (PCA).

Methodology

Data Balancing Techniques

To address the severe class imbalance, the following methods were evaluated:

Random Undersampling

Randomly removes samples from the majority class until the classes become more balanced.

SMOTE

Synthetic Minority Over-sampling Technique generates new synthetic fraud samples by generating new data on the lines between its closest neighbors of the same class rather than duplicating existing ones.

Advantages:

- Increases minority representation
- Preserves majority samples

SMOTE-Tomek

Combines SMOTE with Tomek Links.

- SMOTE creates synthetic fraud samples.
- Tomek Links remove majority-class samples that form a mutual nearest-neighbor pair with a minority-class sample.

This both increases minority representation and cleans noisy decision boundaries.

Cluster Centroids

Compresses the majority class into representative cluster centers.

Feature Analysis

Several feature analysis techniques were used.

Built-In Important Features

Random Forest and Logistic Regression provide built-in important features and coefficients respectively.

SHAP Values

Measure how much each feature contributes to an individual prediction.
Used to determine overall feature importance.

SHAP Interaction Values

Measure how pairs of features interact together to influence predictions.
Useful for identifying important feature relationships.

Pearson Correlation

Measures linear relationships between features.

Spearman Correlation

Measures monotonic relationships, including non-linear trends.

Mutual Information

Measures statistical dependency between variables without assuming linearity.
Useful for detecting complex relationships.

Experiments

The project includes several experiments:

- Comparing balancing methods
- Comparing machine learning models
- Feature importance analysis
- Training size analysis
- Temporal stability analysis
- Recall optimization through threshold tuning

Results

Best Model

model	balancing	class_weight	threshold_mode	precision	recall	f1	pr_auc	roc_auc
AdaBoost	None	None	default	0.705882	0.734694	0.720000	0.724277	0.979230
KNN	None	Balanced	default	1.000000	0.030612	0.059406	0.114919	0.636393
Logistic Regression	None	None	default	0.817073	0.683673	0.744444	0.694752	0.955090
Random Forest	SMOTE_Tomek	Balanced	default	0.831683	0.857143	0.844221	0.873052	0.969262

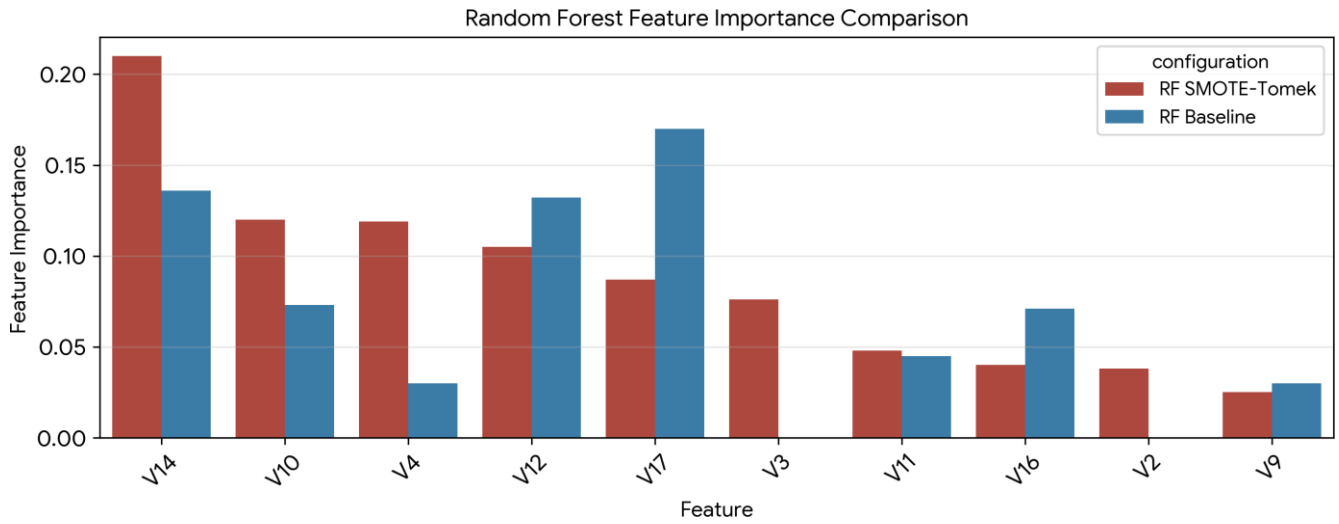
Random Forest consistently achieved the strongest overall performance.

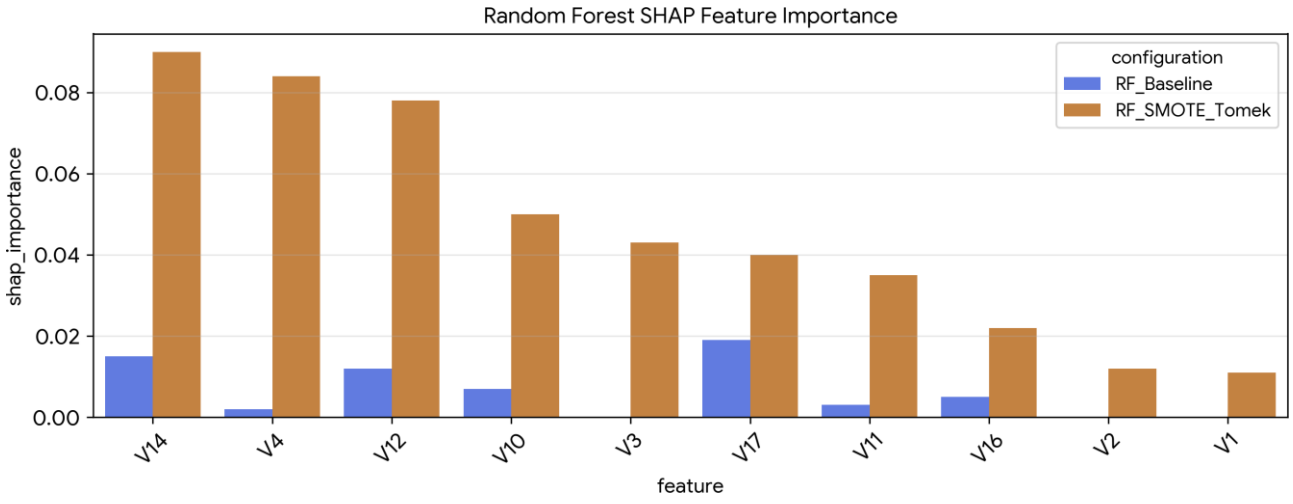
Best Balancing Method

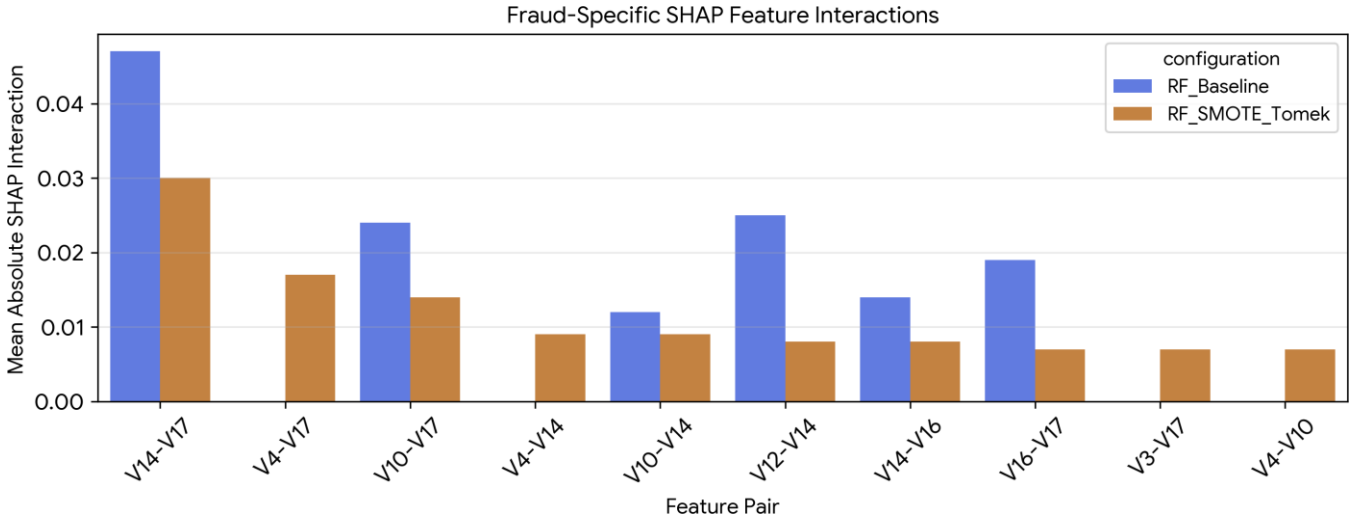
SMOTE-Tomek produced the best trade-off between precision and recall.
It significantly increased fraud detection (Recall) while maintaining a reasonable level of precision.

Most Important Features

The five most influential features were:



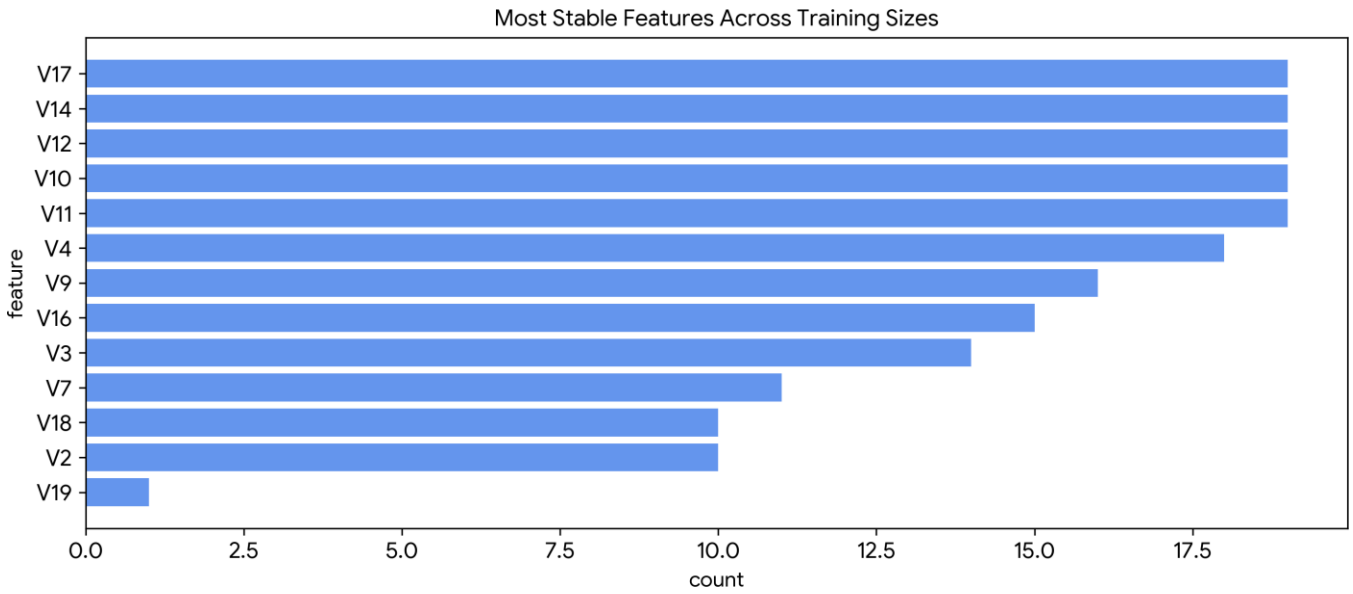




- V17
- V14
- V12
- V10
- V4

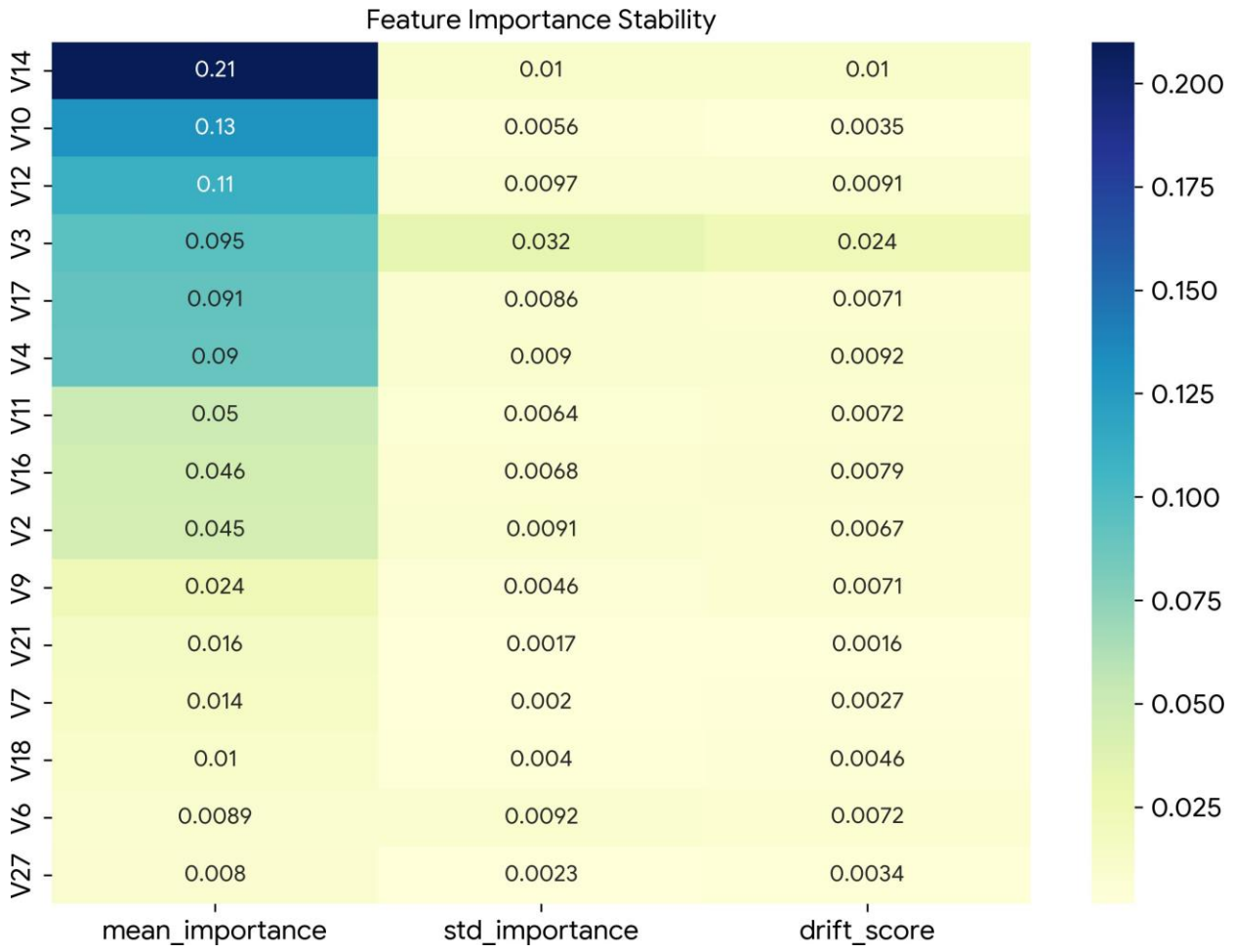
These rankings were supported by both SHAP values and additional feature analysis experiments.

Training Size



Performance improved steadily until approximately **60% of the available training data**.
Beyond this point, additional data resulted in little improvement and, in some cases, slight performance degradation.

Temporal Stability



Random Forest maintained stable performance across different temporal segments of the dataset.
The important features also remained largely consistent over time.

Challenges Encountered

1. Extreme Class Imbalance

Problem

Only 0.172% of transactions are fraudulent, making it difficult for models to learn the minority class.

Solution

Evaluated several balancing methods and found **SMOTE-Tomek** to be the most effective.

---

2. Optimizing Recall

Problem

Fraud detection prioritizes catching fraudulent transactions, making Recall especially important.

Solution

Threshold optimization was used to target a Recall of approximately 0.95.

Outcome

model	balancing	class_weight	threshold_mode	precision	recall	f1	pr_auc	roc_auc
AdaBoost	None	None	recall	0.011106	0.959184	0.021957	0.724277	0.979230
KNN	ClusterCentroids	None	recall	0.001766	0.979592	0.003526	0.008205	0.730227
Logistic Regression	SMOTE	None	recall	0.012648	0.959184	0.024967	0.730241	0.976502
Random Forest	Undersampling	None	recall	0.011229	0.969388	0.022201	0.695286	0.977698

Although Recall increased, Precision dropped substantially, producing too many false positives.

3.Identifying Important Features

Problem

The dataset contains 28 anonymized PCA features whose individual values provide little intuitive meaning.

Solution

Used SHAP values and SHAP interaction values to rank features and analyze their contributions.

---

4. Model Evaluation

Problem

Overall accuracy is misleading for highly imbalanced datasets.

Solution

Focused primarily on the Precision-Recall trade-off, emphasizing high Recall while maintaining acceptable Precision.

Techniques That Worked Well

SMOTE-Tomek

Successfully increased minority-class representation while cleaning ambiguous majority-class samples near the decision boundary. This resulted in consistently higher Recall with only a moderate decrease in Precision, making it the best balancing strategy for this dataset.

SHAP-Based Feature Analysis

SHAP values and SHAP interaction values consistently identified the same important features across multiple experiments, providing interpretable and stable feature importance rankings.

Techniques That Were Less Effective

Recall Optimization

Aggressively lowering the classification threshold achieved very high Recall but caused Precision to deteriorate significantly, leading to an excessive number of false positives.

Random Undersampling and Cluster Centroids

Both methods removed a substantial amount of majority-class information. Given the dataset's extreme imbalance, this loss of information outweighed the benefits of balancing.

KNN and AdaBoost

KNN struggled with the high-dimensional feature space and severe class imbalance, while AdaBoost was less effective at capturing the complex patterns required to distinguish fraudulent transactions. Both models consistently underperformed compared to Random Forest.

Requirements

Typical Python libraries used:

- scikit-learn
- imbalanced-learn
- SHAP
- pandas
- numpy
- matplotlib
- scipy

Dataset

The dataset is available on Kaggle:
https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud/data